

ĐẠI HỌC HUẾ
TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN



BÀI TIỂU LUẬN

HỌC PHẦN: PHÁT TRIỂN ỨNG DỤNG IoT

ĐỀ TÀI:
CẢM BIẾN ĐẾM NGƯỜI TRONG KHÔNG GIAN NHỎ VỚI ESP32
(Dùng cảm biến quang trở đếm lượt ra/vào, hiển thị trên màn hình OLED)

Sinh viên thực hiện: Hồ Tiến Bảo
Lớp học phần: Phát triển ứng dụng IoT – Nhóm 1

Thừa Thiên Huế, năm 2025

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Nghĩa đầy đủ
ESP32	Espressif Systems 32-bit – Vi điều khiển 32-bit tích hợp Wi-Fi/Bluetooth
IoT	Internet of Things – Vạn vật kết nối
LDR	Light Dependent Resistor – Quang trở, điện trở thay đổi theo ánh sáng
GPIO	General Purpose Input/Output – Chân vào/ra đa năng
OLED	Organic Light-Emitting Diode – Màn hình LED hữu cơ tự phát sáng
I2C	Inter-Integrated Circuit – Giao thức truyền thông nối tiếp 2 dây
SDA	Serial Data Line – Đường dữ liệu nối tiếp của I2C
SCL	Serial Clock Line – Đường nhịp đồng hồ của I2C
FSM	Finite State Machine – Máy trạng thái hữu hạn
DO	Digital Output – Chân tín hiệu số đầu ra của module cảm biến quang trở
PlatformIO	Hệ sinh thái lập trình nhúng mã nguồn mở, tích hợp VS Code
Wokwi	Nền tảng mô phỏng phần cứng nhúng trực tuyến (wokwi.com)
ADC	Analog to Digital Converter – Bộ chuyển đổi tương tự sang số
PWM	Pulse Width Modulation – Điều chế độ rộng xung

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	1
MỞ ĐẦU	3
1. Lý do được giao đề tài	3
2. Mục tiêu nghiên cứu	3
3. Đối tượng và phạm vi	4
4. Phương pháp thực hiện	4
PHẦN NỘI DUNG	4
Chương 1: Tổng quan lý thuyết	4
1.1. Internet of Things và xu hướng ứng dụng	4
1.2. Vi điều khiển ESP32 – Thông số kỹ thuật	5
1.3. Cảm biến quang trở – Nguyên lý và thông số	6
1.4. Màn hình OLED SSD1306	8
1.5. Giao thức I2C	9
1.6. Nền tảng mô phỏng Wokwi	9
Chương 2: Thiết kế hệ thống	10
2.1. Yêu cầu chức năng	10
2.2. Sơ đồ khối hệ thống	10
2.3. Nguyên lý đếm chiều ra/vào (FSM)	11
2.4. Sơ đồ kết nối phần cứng (diagram.json)	12
2.5. Danh sách linh kiện	12
Chương 3: Triển khai và lập trình	13
3.1. Môi trường phát triển VS Code + PlatformIO	13
3.2. Cấu trúc dự án	13
3.3. File sơ đồ mạch diagram.json	14
3.4. Code chương trình main.cpp	15
3.5. Cách vận hành trên Wokwi	18
Chương 4: Kết quả và đánh giá	19
4.1. Kết quả mô phỏng	19
4.2. Nhận xét và đánh giá	20
KẾT LUẬN VÀ KIẾN NGHỊ	21
TÀI LIỆU THAM KHẢO	22

MỞ ĐẦU

1. Lý do được giao đề tài

Trong học kỳ này, môn học "Phát triển ứng dụng IoT" tổ chức đánh giá cuối kỳ theo hình thức tiểu luận cá nhân thay vì thi viết tập trung. Phòng Khảo thí và Bảo đảm Chất lượng Giáo dục phân công cho mỗi sinh viên trong lớp một chủ đề nghiên cứu riêng biệt, nhằm đảm bảo sự đa dạng về nội dung và tính độc lập trong quá trình thực hiện, tránh trùng lặp đề tài giữa các sinh viên.

Theo danh sách phân công chính thức, em – sinh viên Hồ Tiến Bảo – được giao đề tài: "Cảm biến đếm người trong không gian nhỏ với ESP32 – dùng cảm biến quang trở mô phỏng che tia để đếm lượt ra/vào, hiển thị trên màn hình". Mặc dù đây là chủ đề do nhà trường chỉ định, nhưng khi tìm hiểu sâu hơn, em nhận thấy đề tài này khá thú vị và rất gần với thực tế: các hệ thống đếm người tự động đang được sử dụng ngày càng nhiều ở các siêu thị, thư viện, phòng thi... để kiểm soát lượng người ra vào một cách tự động mà không cần nhân viên trực tiếp đứng đếm.

Vì vậy, dù đề tài là do nhà trường phân công, em đã chủ động tìm hiểu tài liệu, nghiên cứu nguyên lý hoạt động của các linh kiện và thực hành lập trình trên môi trường VS Code với PlatformIO, đồng thời mô phỏng toàn bộ mạch điện trên nền tảng Wokwi trực tuyến.

2. Mục tiêu nghiên cứu

- Hiểu được cấu tạo và nguyên lý hoạt động của cảm biến quang trở (photoresistor) dùng theo nguyên lý che tia sáng trong ứng dụng phát hiện người ra/vào.
- Nắm vững các thông số kỹ thuật của ESP32, OLED SSD1306 và cách lập trình trên Arduino Framework.
- Xây dựng thuật toán máy trạng thái (FSM) phân biệt chiều vào/ra chỉ với hai cảm biến đơn giản.
- Thiết kế sơ đồ mạch mô phỏng (diagram.json) và lập trình hoàn chỉnh (main.cpp) trên Wokwi + VS Code.
- Rèn luyện kỹ năng đọc datasheet, tìm hiểu tài liệu kỹ thuật và ứng dụng kiến thức học phần vào thực tiễn.

3. Đối tượng và phạm vi

Đề tài tập trung vào mô phỏng hệ thống trên Wokwi, không yêu cầu triển khai phần cứng thực tế. Đối tượng nghiên cứu gồm: vi điều khiển ESP32 DevKit C V4, hai module cảm biến quang trở (wokwi-photoresistor-sensor) mô phỏng nguyên lý che tia sáng, màn hình OLED

SSD1306 128×64 và đèn LED báo hiệu. Phạm vi ứng dụng là không gian nhỏ có một cửa ra vào duy nhất.

4. Phương pháp thực hiện

- Tìm hiểu lý thuyết: Đọc datasheet ESP32, SSD1306, xem tài liệu hướng dẫn Wokwi và các bài tutorial về cảm biến quang trở ứng dụng đếm người trên GitHub và Arduino Forum.
- Thiết kế mạch: Xây dựng file diagram.json mô tả cấu hình linh kiện và kết nối dây trên Wokwi.
- Lập trình: Viết code C++ trong VS Code với PlatformIO, thực hiện mô phỏng và gỡ lỗi qua Wokwi.
- Kiểm thử: Thử nghiệm các tình huống: người đi vào, đi ra, đứng giữa cửa, đi không qua hẳn... và đánh giá kết quả qua Serial Monitor.

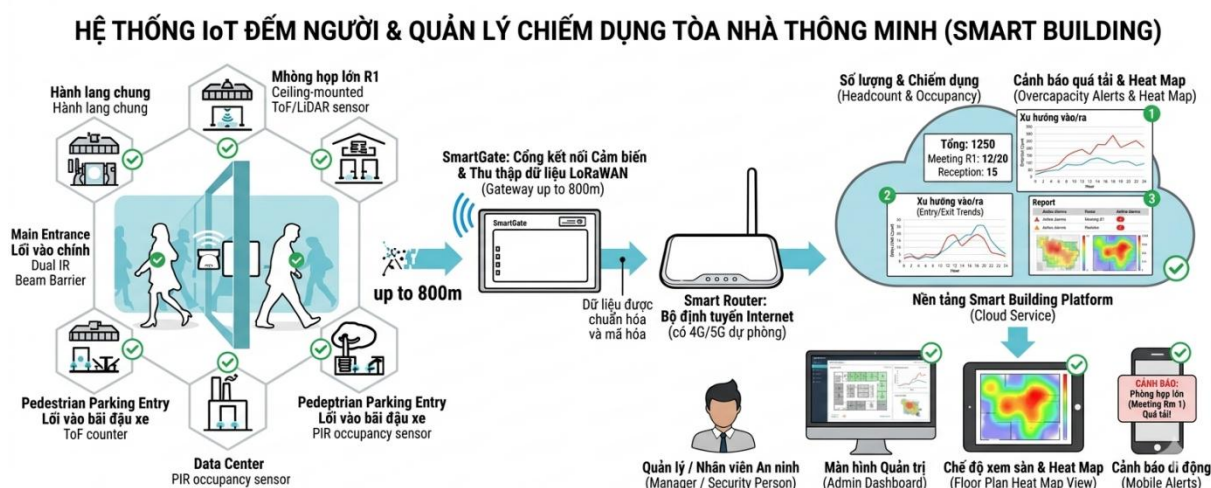
PHẦN NỘI DUNG

Chương 1: Tổng quan lý thuyết

1.1. Internet of Things và xu hướng ứng dụng:

Internet of Things (IoT) là khái niệm mô tả một mạng lưới rộng lớn các thiết bị vật lý – từ đồng hồ thông minh, cảm biến nhiệt độ cho đến các hệ thống điều khiển tòa nhà – được kết nối với nhau và với internet, có khả năng thu thập, truyền tải và xử lý dữ liệu mà không cần hoặc chỉ cần rất ít sự can thiệp của con người. Theo Liên minh Viễn thông Quốc tế (ITU-T Y.2060), IoT được định nghĩa là "cơ sở hạ tầng toàn cầu cho xã hội thông tin, cho phép các dịch vụ tiên tiến thông qua việc kết nối các vật thể dựa trên công nghệ thông tin và truyền thông hiện có và đang phát triển" [1].

Trong lĩnh vực quản lý không gian và tòa nhà thông minh (Smart Building), các hệ thống đếm người tự động (People Counting System) ngày càng được ứng dụng rộng rãi. Thay vì để một nhân viên ngồi đếm tay, các cảm biến tự động thực hiện nhiệm vụ này 24/7, cung cấp dữ liệu theo thời gian thực để: kiểm soát mật độ người theo quy định phòng dịch, tự động bật/tắt điều hòa và đèn khi không có người, thống kê lưu lượng khách hàng theo giờ/ngày cho các cửa hàng bán lẻ.



Hình 1: Ứng dụng IoT đếm người trong tòa nhà thông minh

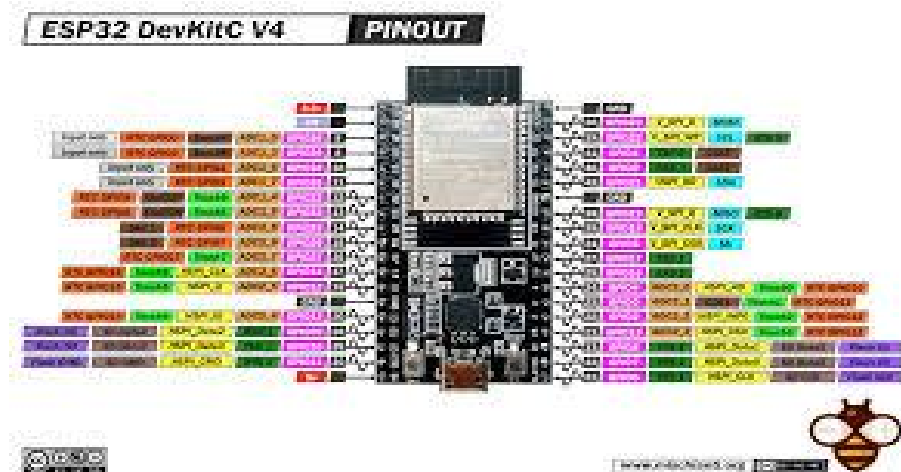
1.2. Vi điều khiển ESP32 – Thông số kỹ thuật

ESP32 là vi điều khiển do Espressif Systems (Thượng Hải, Trung Quốc) thiết kế, ra mắt vào cuối năm 2016. Đây là thế hệ kế tiếp của ESP8266 với nhiều cải tiến đáng kể, đặc biệt là lõi kép và khả năng kết nối không dây tích hợp. Chip này nhanh chóng trở thành lựa chọn hàng đầu của cộng đồng maker và các kỹ sư IoT nhờ giá thành rất thấp (khoảng 3–8 USD/module) trong khi tính năng không kém gì các vi điều khiển đắt tiền hơn [2].

Trong dự án này em dùng module ESP32 DevKit C V4 – phiên bản phổ biến nhất, có sẵn trong Wokwi. Dưới đây là các thông số kỹ thuật quan trọng:

Bảng 1: Thông số kỹ thuật ESP32 DevKit C V4

Thông số	Giá trị / Mô tả
CPU	Xtensa LX6 dual-core, tốc độ lên đến 240 MHz
RAM (SRAM)	520 KB – dùng cho stack, heap, biến toàn cục
Flash	4 MB (module DevKit thông dụng)
Wi-Fi	802.11 b/g/n, tần số 2.4 GHz, tốc độ đến 150 Mbps
Bluetooth	Classic 4.2 + BLE (Bluetooth Low Energy)
Số chân GPIO	34 chân (18 ADC 12-bit, 2 DAC 8-bit, 16 PWM)
Giao thức	I2C, SPI, UART, I2S, CAN, SDIO
Điện áp GPIO	3.3V (không chịu 5V trực tiếp!)
Điện áp cấp nguồn	5V qua chân VIN hoặc USB
Tiêu thụ điện (hoạt động)	Trung bình ~80–240 mA tùy tác vụ
Tiêu thụ điện (deep sleep)	~10 μ A – rất tiết kiệm điện
Nhiệt độ hoạt động	-40°C đến +85°C



Hình 2: Module ESP32 DevKit C V4 và sơ đồ chân (pinout)

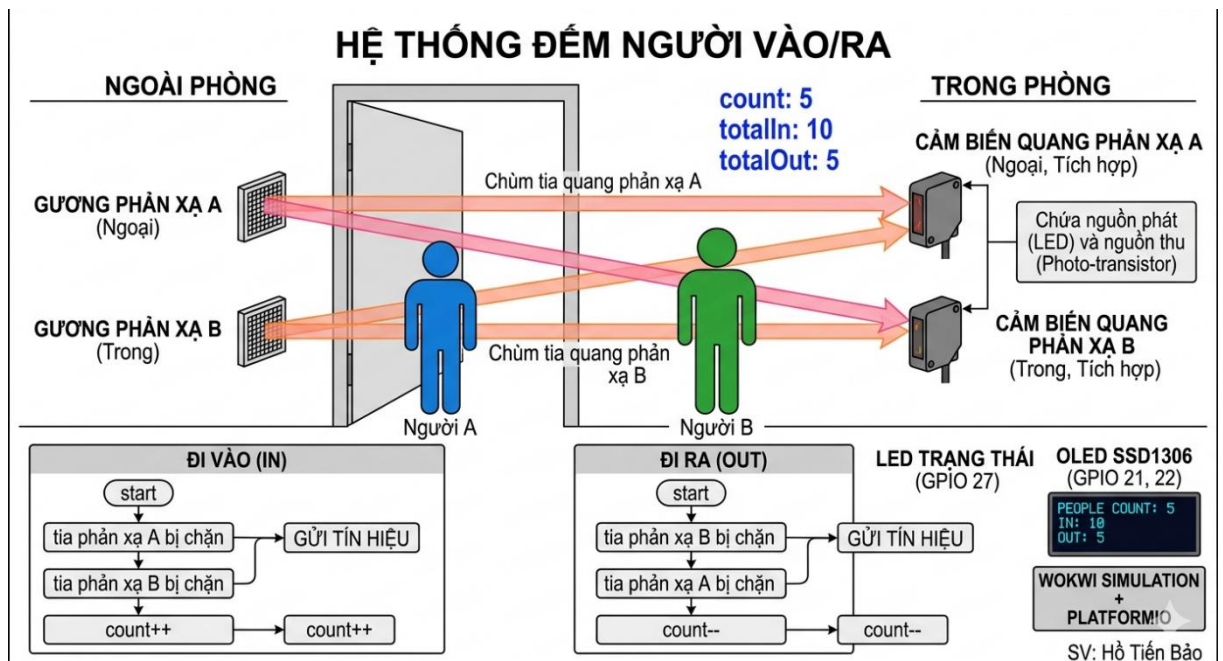
Một điểm rất quan trọng khi làm việc với ESP32 là các chân GPIO chỉ chịu được mức logic 3.3V. Nếu nối trực tiếp với thiết bị 5V mà không qua mạch chuyển mức sẽ làm hỏng chip. Trong dự án này, cảm biến quang trở được cấp 3.3V và đầu ra tín hiệu DO kết nối thẳng vào GPIO của ESP32, hoàn toàn tương thích về mức điện áp.

1.3. Cảm biến quang trở – Nguyên lý và thông số

Quang trở (photoresistor hay LDR – Light Dependent Resistor) là linh kiện thụ động có điện trở thay đổi theo cường độ ánh sáng chiếu vào. Khi ánh sáng mạnh, điện trở giảm xuống còn vài trăm Ω ; khi tối hoàn toàn, điện trở tăng lên hàng trăm k Ω . Ưu điểm nổi bật của quang trở là giá thành rất rẻ, dễ sử dụng, không cần nguồn phát riêng và có thể kết hợp với mạch so sánh để tạo tín hiệu số HIGH/LOW.

Trong hệ thống đếm người trên Wokwi, component được sử dụng là wokwi-photoresistor-sensor – module cảm biến quang trở tích hợp sẵn mạch so sánh, gồm 3 chân: VCC, GND và DO (Digital Output). Nguyên lý hoạt động mô phỏng che tia sáng:

- Khi có ánh sáng (không bị che): quang trở có điện trở thấp $\rightarrow$ mạch so sánh kéo DO xuống LOW. Đây là trạng thái bình thường khi không có người đứng chắn.
- Khi bị che (người đứng chắn): quang trở có điện trở cao $\rightarrow$ mạch so sánh đẩy DO lên HIGH. Trong Wokwi, người dùng click chuột vào component để chuyển trạng thái tối/sáng, mô phỏng hành động che/thả cảm biến.



Hình 3: Cấu tạo và nguyên lý hoạt động cảm biến quang trở (wokwi-photoresistor-sensor) mô phỏng che tia

Bảng 2: Thông số kỹ thuật module cảm biến quang trở wokwi-photoresistor-sensor sử dụng trong dự án

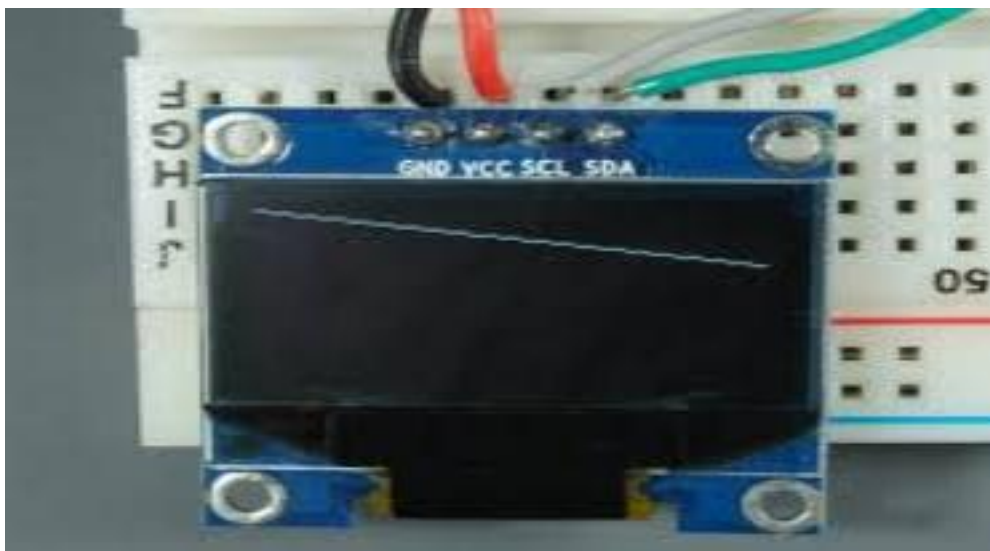
Thông số	Giá trị	Ghi chú
Loại linh kiện	Quang trở (LDR) + mạch so sánh	Tích hợp sẵn trong 1 module Wokwi
Điện áp hoạt động (VCC)	3.3V – 5V	Dùng 3.3V để tương thích trực tiếp GPIO ESP32
Chân tín hiệu ra (DO)	0V (LOW) hoặc 3.3V	LOW = có ánh sáng, HIGH = bị

	(HIGH)	che
Trạng thái mặc định	LOW (có ánh sáng)	Không có người → sensor không bị che
Trạng thái khi bị che	HIGH (tối)	Người đứng chắn → sensor bị che tối
Thời gian phản hồi	< 10 ms	Đầu ra HIGH khi tia bị chặn
Điện trở (sáng)	~1 kΩ	Tùy loại LDR sử dụng
Điện trở (tối)	> 100 kΩ	Khi bị che hoàn toàn
Thời gian phản hồi	Component Wokwi	wokwi-photoresistor-sensor
Chân GPIO kết nối	GPIO18 (A), GPIO19 (B)	Không cần thêm linh kiện phụ

1.4. Màn hình OLED SSD1306

SSD1306 là chip điều khiển màn hình OLED (Organic Light-Emitting Diode) được sản xuất bởi Solomon Systech. Module phổ biến nhất có kích thước 128×64 pixels, giao tiếp I2C hoặc SPI. Ưu điểm nổi bật: không cần đèn nền (tự phát sáng), độ tương phản rất cao, tiêu thụ điện thấp (~20 mA khi sáng toàn màn), hoạt động ở 3.3V và 5V, giá thành rẻ (~20.000 VNĐ).

Trong dự án, thư viện Adafruit SSD1306 (kết hợp với Adafruit GFX) được sử dụng để điều khiển màn hình. Địa chỉ I2C mặc định là 0x3C. Màn hình hiển thị 3 chế độ: Boot Screen (khi khởi động), Main Display (số người hiện tại + thống kê), Event Screen (thông báo khi có người vào/ra).



Hình 4: Module OLED SSD1306 128×64 và cách kết nối I2C

1.5. Giao thức I2C

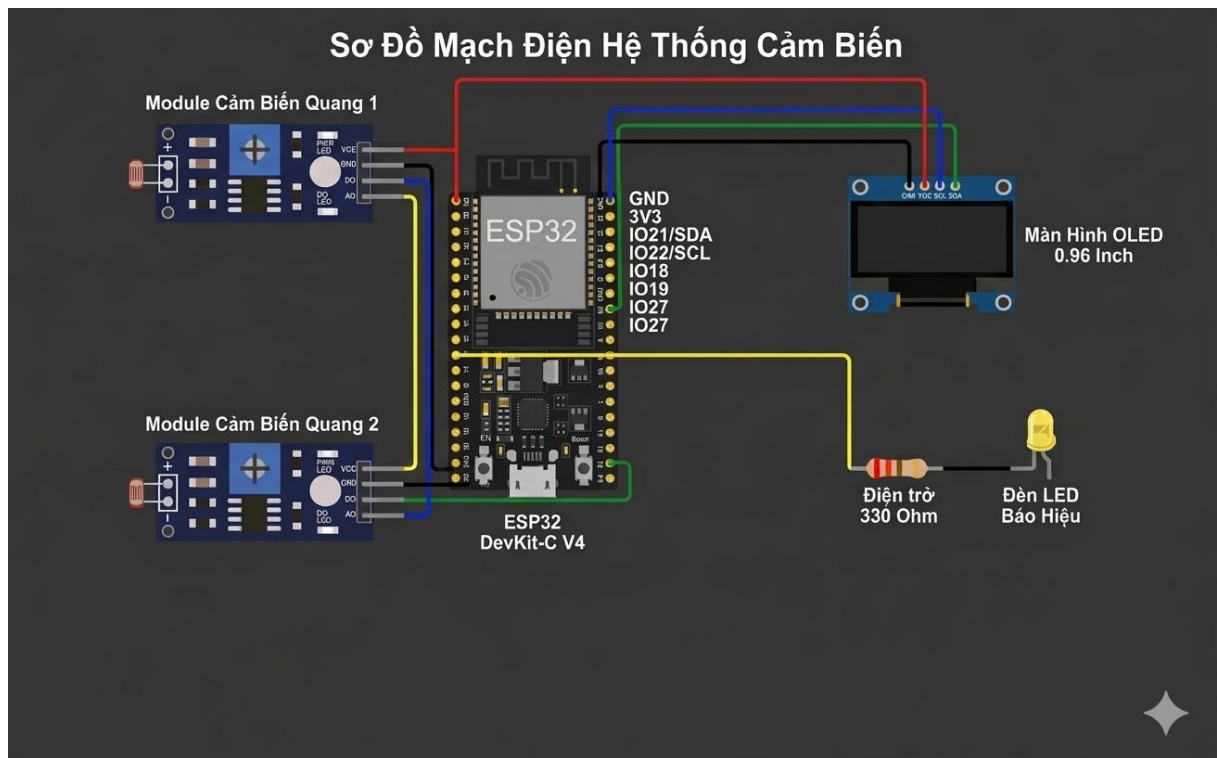
I2C (Inter-Integrated Circuit) là giao thức truyền thông nối tiếp đồng bộ 2 dây do Philips (nay là NXP) phát triển năm 1982. I2C chỉ cần 2 dây: SDA (Serial Data) truyền dữ liệu và SCL

(Serial Clock) cung cấp nhịp đồng hồ. Mỗi thiết bị trên bus có địa chỉ 7-bit duy nhất. Trong dự án này, OLED SSD1306 được kết nối I2C với ESP32 qua GPIO21 (SDA) và GPIO22 (SCL) ở tốc độ 100 kHz.

1.6. Nền tảng mô phỏng Wokwi

Wokwi (wokwi.com) là nền tảng mô phỏng phần cứng nhúng trực tuyến, cho phép lập trình và chạy thử nghiệm mạch điện Arduino/ESP32 ngay trên trình duyệt mà không cần phần cứng thực. Wokwi hỗ trợ tích hợp với VS Code thông qua extension "Wokwi for VS Code", cho phép build firmware bằng PlatformIO rồi mô phỏng trực tiếp từ IDE mà không cần nạp lên board thật [6].

File diagram.json mô tả toàn bộ sơ đồ mạch (linh kiện, vị trí, kết nối dây) theo định dạng JSON. File wokwi.toml trỏ đến file firmware.elf đã biên dịch để Wokwi chạy mô phỏng.



Hình 5: Giao diện Wokwi mô phỏng mạch đếm người

Chương 2: Thiết kế hệ thống

2.1. Yêu cầu chức năng

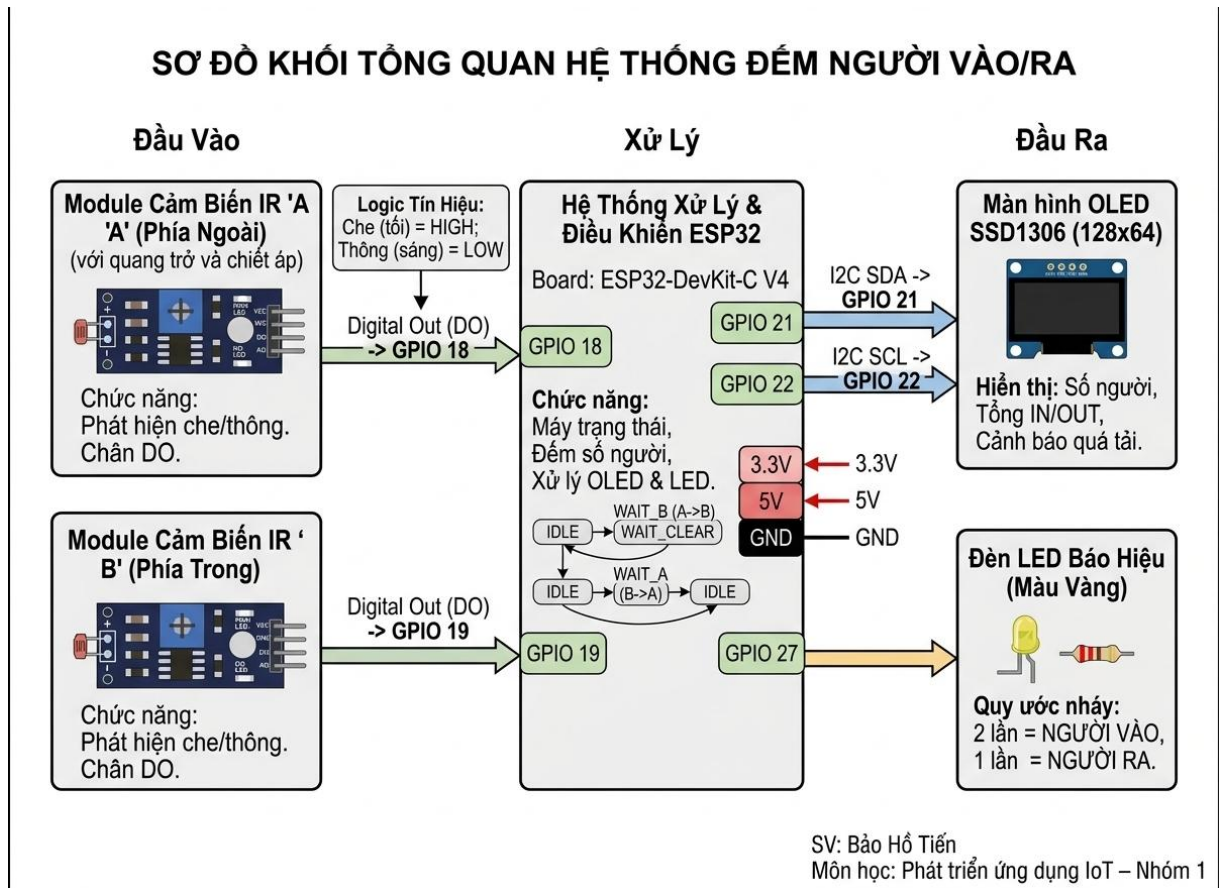
Hệ thống cần đáp ứng các yêu cầu chức năng sau:

- Phát hiện người đi qua cửa theo hai chiều: VÀO (từ ngoài vào trong) và RA (từ trong ra ngoài).
- Đếm chính xác số người hiện đang ở trong phòng (không âm).
- Thống kê tổng số lượt vào và lượt ra từ khi khởi động.
- Hiển thị thông tin trên màn hình OLED: số người hiện tại, tổng vào, tổng ra.
- Báo hiệu bằng LED khi có người vào (nháy 2 lần) hoặc ra (nháy 1 lần).
- Cảnh báo bằng màn hình khi số người vượt ngưỡng $MAX_PEOPLE = 30$.
- Xử lý các tình huống đặc biệt: người quay lại, người đứng giữa cửa quá lâu (timeout 4 giây).
- In log chi tiết mỗi sự kiện ra Serial Monitor (115200 baud).

2.2. Sơ đồ khối hệ thống

Hệ thống gồm 4 khối chính:

- Khối cảm biến: 2 module quang trở (wokwi-photoresistor-sensor) đặt tại vị trí A (ngoài cửa) và B (trong cửa), mô phỏng nguyên lý người đi qua che/thả ánh sáng cảm biến.
- Khối xử lý: ESP32 DevKit C V4 chạy firmware PlatformIO/Arduino, thực hiện thuật toán FSM đếm người.
- Khối hiển thị: Màn hình OLED SSD1306 128×64 kết nối I2C, hiển thị số người và thống kê.
- Khối báo hiệu: LED vàng 5mm kết nối qua điện trở 220Ω, nháy khi có sự kiện.



Hình 6: Sơ đồ khối tổng quan hệ thống

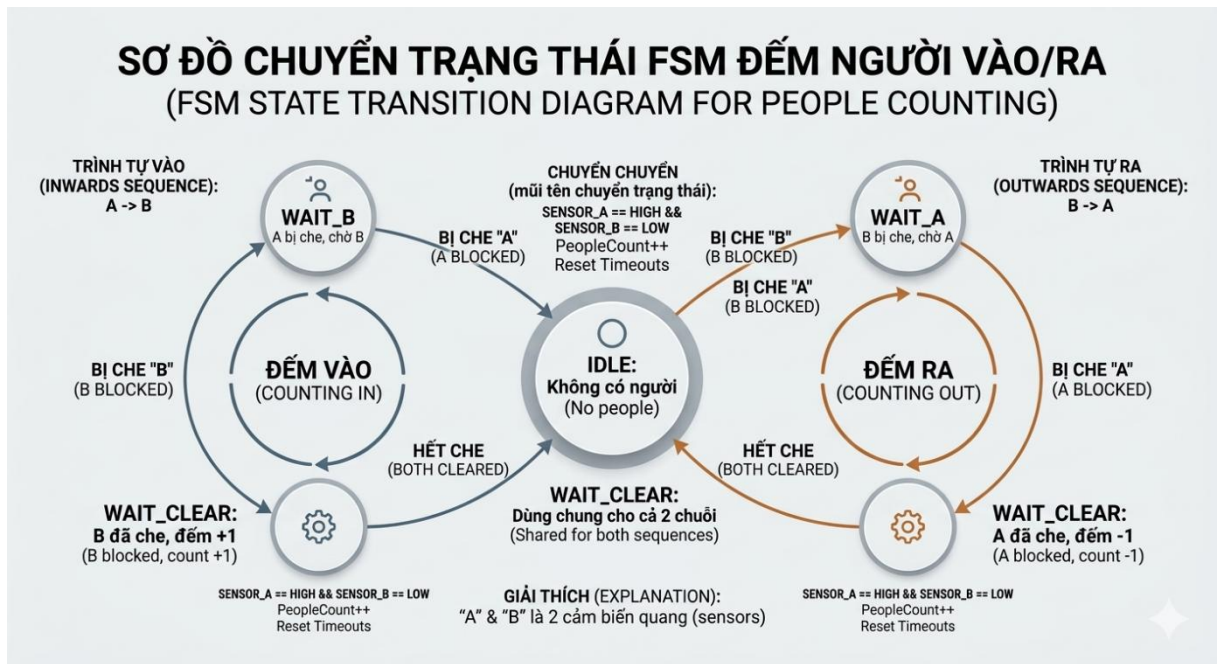
2.3. Nguyên lý đếm chiều ra/vào (FSM)

Máy trạng thái hữu hạn (Finite State Machine – FSM) là thuật toán cốt lõi của hệ thống. FSM có 4 trạng thái:

- IDLE:** Không có ai – cả 2 sensor đều có ánh sáng (DO = LOW, CLEAR). Đây là trạng thái chờ.
- WAIT_B:** Sensor A bị chặn trước → khả năng người đang đi VÀO, chờ Sensor B bị chặn để xác nhận.
- WAIT_A:** Sensor B bị chặn trước → khả năng người đang đi RA, chờ Sensor A bị chặn để xác nhận.
- WAIT_CLEAR:** Đã đếm xong một lượt, chờ cả 2 sensor CLEAR (người đi qua hoàn toàn) để tránh đếm đúp.

Cơ chế Timeout: Nếu ở trạng thái không phải IDLE quá 4000ms mà không hoàn thành lượt, hệ thống tự reset về IDLE.

Cơ chế Debounce: Mỗi lần lặp đọc cảm biến được cách nhau tối thiểu 40ms để lọc nhiễu tín hiệu.



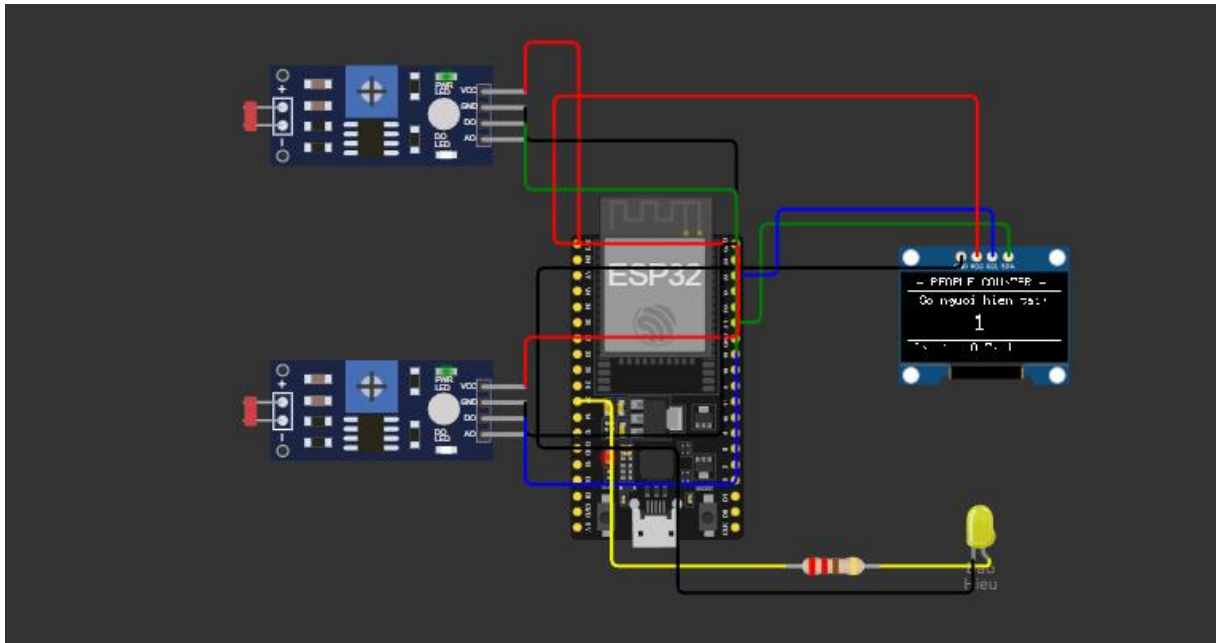
Hình 7: Sơ đồ máy trạng thái FSM phân biệt chiều vào/ra

2.4. Sơ đồ kết nối phần cứng (diagram.json)

Bảng dưới đây liệt kê chi tiết các kết nối pin giữa ESP32 và các linh kiện trong mạch:

Bảng 3: Bảng kết nối chân ESP32 với linh kiện

Chân ESP32	Kết nối đến	Chức năng	Ghi chú
GPIO 18	Chân DO – Sensor A (quang trở ngoài cửa)	Digital Input	Cảm biến ngoài cửa (Sensor A)
GPIO 19	Chân DO – Sensor B (quang trở trong cửa)	Digital Input	Cảm biến trong cửa (Sensor B)
GPIO 21	SDA màn hình OLED	I2C Data	Dữ liệu giao thức I2C
GPIO 22	SCL màn hình OLED	I2C Clock	Nhịp đồng hồ giao thức I2C
GPIO 27	Anod LED qua R220Ω	Digital Output	LED báo hiệu màu vàng
3.3V	VCC OLED + Pull-up R10kΩ	Power	Nguồn 3.3V cho OLED, Sensor A và B
5V	Không sử dụng (không có IR LED)	Power	Không sử dụng trong mạch này
GND	GND Sensor A, GND Sensor B, GND OLED, Cathode LED	Ground	Chung mass toàn mạch



Hình 8: Sơ đồ mạch chi tiết trên Wokwi (ảnh chụp màn hình)

2.5. Danh sách linh kiện

Bảng 4: Danh sách linh kiện và chi phí ước tính

STT	Tên linh kiện	SL	Mô tả / Chức năng	Giá ước tính
1	ESP32 DevKit C V4	1	Vi điều khiển chính 240MHz, Wi-Fi, BLE	~80.000 VNĐ
2	Module quang trở LDR (GL5537)	2	Thay đổi điện trở theo ánh sáng, xuất tín hiệu số DO	~5.000 VNĐ/cặp
3	Mạch so sánh LM393	2	Thu tia IR, dẫn khi nhận tia	Tích hợp trong module quang trở
4	Điện trở 220Ω	2	Phân áp cho quang trở trong mạch so sánh	~500 VNĐ
5	Điện trở 10kΩ	2	Không còn sử dụng (gộp vào module)	~500 VNĐ
6	Màn hình OLED SSD1306 128×64	1	Hiển thị số người và thống kê qua I2C	~20.000 VNĐ
7	LED vàng 5mm	1	Báo hiệu khi có người vào/ra	~1.000 VNĐ
8	Điện trở 220Ω (LED)	1	Hạn dòng LED báo hiệu	~500 VNĐ
9	Dây kết nối (jumper)	Nhiều	Đấu nối trên breadboard	~5.000 VNĐ
10	Breadboard 830 lỗ	1	Bảng thử mạch không hàn	~15.000 VNĐ

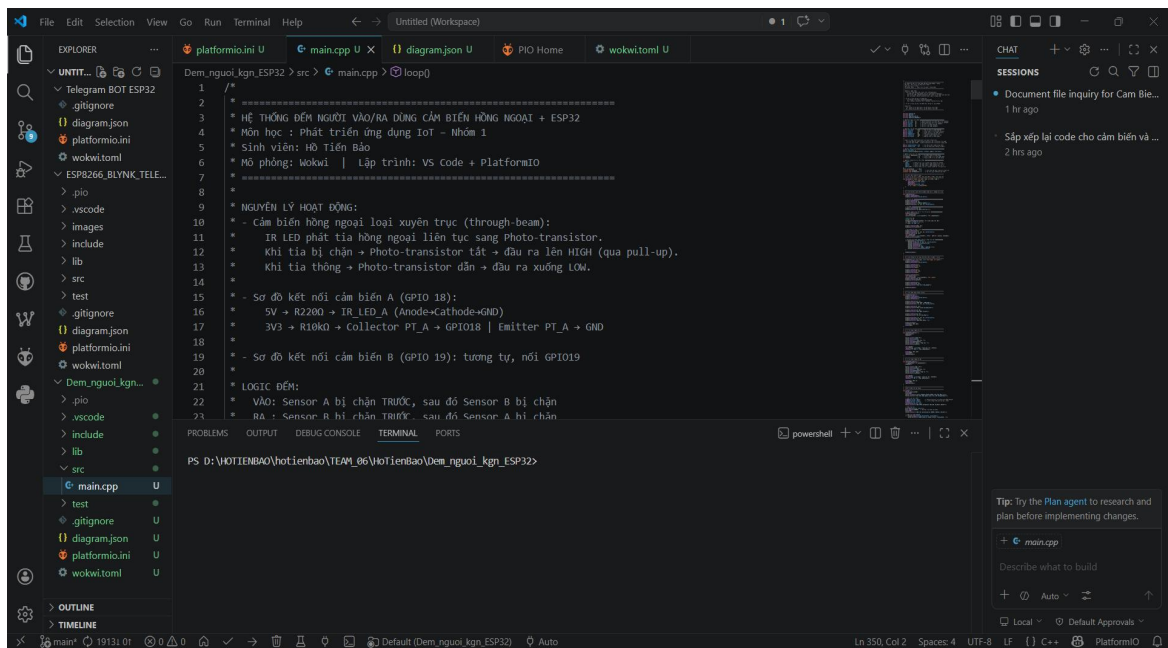
Chương 3: Triển khai và lập trình

3.1. Môi trường phát triển VS Code + PlatformIO

Dự án được phát triển trên VS Code với extension PlatformIO IDE. PlatformIO là hệ sinh thái lập trình nhúng mã nguồn mở, tích hợp trực tiếp vào VS Code, hỗ trợ hơn 1000 board khác nhau và quản lý thư viện tự động [5].

Cài đặt môi trường:

- Bước 1: Cài VS Code từ <https://code.visualstudio.com>
- Bước 2: Trong VS Code, vào Extensions (Ctrl+Shift+X), tìm và cài "PlatformIO IDE"
- Bước 3: Cài extension "Wokwi for VS Code" (Wokwi Simulator)
- Bước 4: Mở thư mục dự án trong VS Code, PlatformIO tự động nhận diện và cài thư viện



Hình 9: Giao diện VS Code + PlatformIO mở project đếm người

3.2. Cấu trúc dự án

Cấu trúc thư mục dự án PlatformIO chuẩn:



3.3. File cấu hình platformio.ini

File platformio.ini khai báo board, framework và thư viện phụ thuộc:

```
Dem_nguoi_kgn_ESP32 > platformio.ini
9  board = esp32dev
10 framework = arduino
11 monitor_speed = 115200
12 lib_deps =
13     adafruit/Adafruit SSD1306 @ ^2.5.7
14     adafruit/Adafruit GFX Library @ ^1.11.5
```

3.4. Code chương trình main.cpp

Code được tổ chức thành các module chức năng rõ ràng. Phần này mô tả chi tiết các phần quan trọng của code:

a) Định nghĩa chân GPIO và hằng số:

Sử dụng `#define` để khai báo các chân kết nối: `SENSOR_A_PIN = 18` (quang trở A – ngoài cửa), `SENSOR_B_PIN = 19` (quang trở B – trong cửa), `LED_PIN = 27`, `OLED_SDA = 21`, `OLED_SCL = 22`. Hằng `SENSOR_BLOCKED = HIGH` và `SENSOR_CLEAR = LOW` theo logic module quang trở: HIGH khi bị che tối, LOW khi có ánh sáng.

b) Máy trạng thái FSM (enum State):

Bốn trạng thái `IDLE`, `WAIT_B`, `WAIT_A`, `WAIT_CLEAR` được mô hình hóa bằng enum và xử lý trong câu lệnh `switch-case` trong hàm `loop()`.

c) Hàm `handlePersonEnter()` và `handlePersonExit()`:

Hàm `handlePersonEnter()` tăng `peopleCount` và `totalIn`, in log ra Serial, hiển thị màn hình sự kiện, nháy LED 2 lần rồi cập nhật màn hình chính. Hàm `handlePersonExit()` tương tự nhưng giảm `peopleCount` (không âm) và tăng `totalOut`, nháy LED 1 lần.

d) Hàm `updateDisplay()`:

Vẽ giao diện màn hình OLED với tiêu đề `"= PEOPLE COUNTER ="`, số người hiện tại cỡ chữ lớn (`TextSize=2`) căn giữa, đường kẻ phân cách, thống kê IN/OUT ở phía dưới. Khi `peopleCount ≥ 30`, vùng tiêu đề đổi màu (`fillRect` trắng, chữ đen) để cảnh báo.

e) Cơ chế Debounce và Timeout:

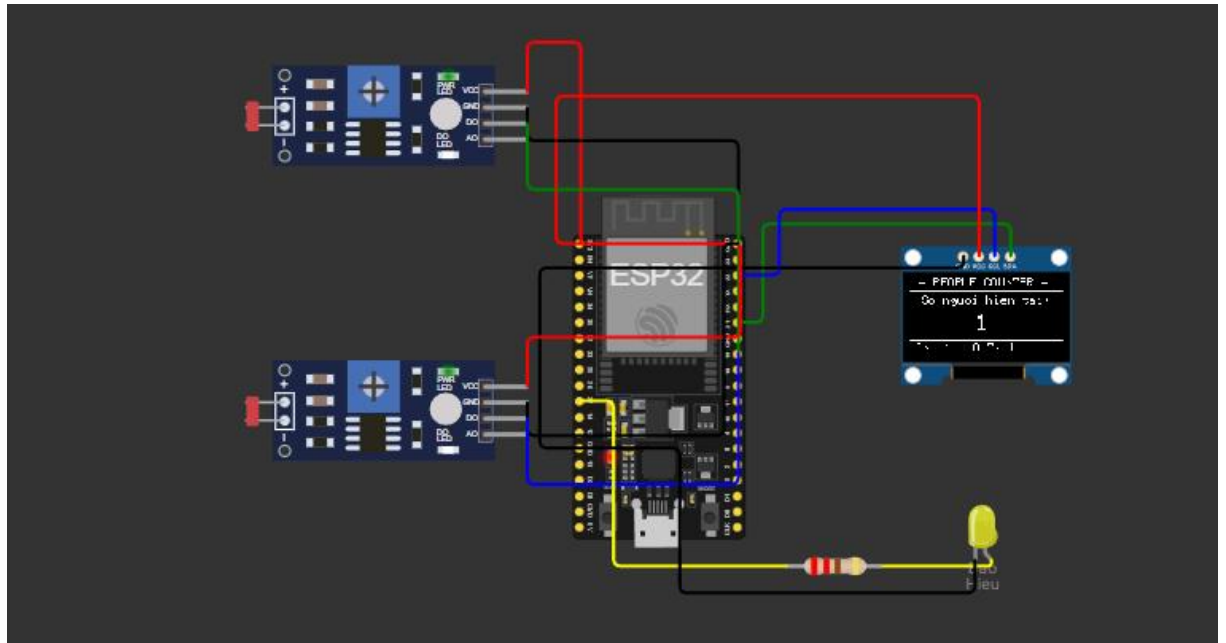
Debounce 40ms: kiểm tra `millis() - lastDebounce < DEBOUNCE_MS` ở đầu loop, bỏ qua nếu chưa đủ thời gian. Timeout 4000ms: nếu `currentState != IDLE` và thời gian vào trạng thái vượt quá 4 giây, tự động reset về IDLE.

Do code khá dài (230+ dòng), em trình bày logic chính ở trên. Toàn bộ code có thể xem trong file `src/main.cpp` của dự án.

3.5. Cách vận hành trên Wokwi

Để chạy mô phỏng trên Wokwi từ VS Code:

- Bước 1: Mở thư mục dự án trong VS Code (File → Open Folder).
- Bước 2: Build firmware: nhấn `Ctrl+Alt+B` hoặc click nút Build của PlatformIO ở thanh Status Bar bên dưới.
- Bước 3: Sau khi build thành công ("Success"), nhấn `F1` → "Wokwi: Start Simulator" để mở Wokwi.
- Bước 4: Trong Wokwi, quan sát màn hình OLED hiển thị Boot Screen rồi chuyển sang màn hình chính.
- Bước 5: Tương tác với cảm biến quang trở: click chuột vào `sensor_a` hoặc `sensor_b` trên sơ đồ Wokwi để chuyển trạng thái tối/sáng, mô phỏng người đứng che/bỏ tay trước cảm biến.
- Bước 6: Quan sát Serial Monitor bên dưới (hoặc mở trong VS Code) để theo dõi log sự kiện.



Hình 11: Wokwi đang chạy mô phỏng – OLED hiển thị số người hiện tại

Chương 4: Kết quả và đánh giá

4.1. Kết quả mô phỏng

Sau khi hoàn thiện code và sơ đồ mạch, em tiến hành kiểm thử bằng cách mô phỏng nhiều tình huống khác nhau trên Wokwi. Kết quả tổng hợp:

Bảng 5: Kết quả kiểm thử các tình huống

TH	Thao tác mô phỏng	Kỳ vọng	Thực tế	Nhận xét
1	Che A → thả A → che B → thả B	Đếm VÀO +1	Đúng ✓	OLED cập nhật, LED nhấp 2 lần, Serial log chính xác
2	Che B → thả B → che A → thả A	Đếm RA -1	Đúng ✓	Số không âm dù count=0, log hiển thị đúng
3	Che A → chờ >4 giây → thả A	Timeout, reset IDLE	Đúng ✓	Serial in [TIMEOUT], màn hình về trạng thái chính
4	Che A rồi thả ngay, không qua B	Hủy bỏ	Đúng ✓	Không đếm, trạng thái về IDLE
5	Đếm liên tục đến 30 người	Cảnh báo đông	Đúng ✓	OLED hiển thị "!! DANG - CANH BAO !!", màu đảo
6	Mô phỏng 5 người vào rồi 5 người ra	IN=5, OUT=5, count=0	Đúng ✓	Số liệu thống kê chính xác hoàn toàn

```

He thong san sang!
[FSM] B bi che -> Cho A (Dang Ra)
Qua thoi gian, huy bo luot di.
[FSM] B bi che -> Cho A (Dang Ra)
Qua thoi gian, huy bo luot di.
[FSM] B bi che -> Cho A (Dang Ra)
=> CO NGUOI RA. Tong ra: 1, Hien tai: 0
Qua thoi gian, huy bo luot di.

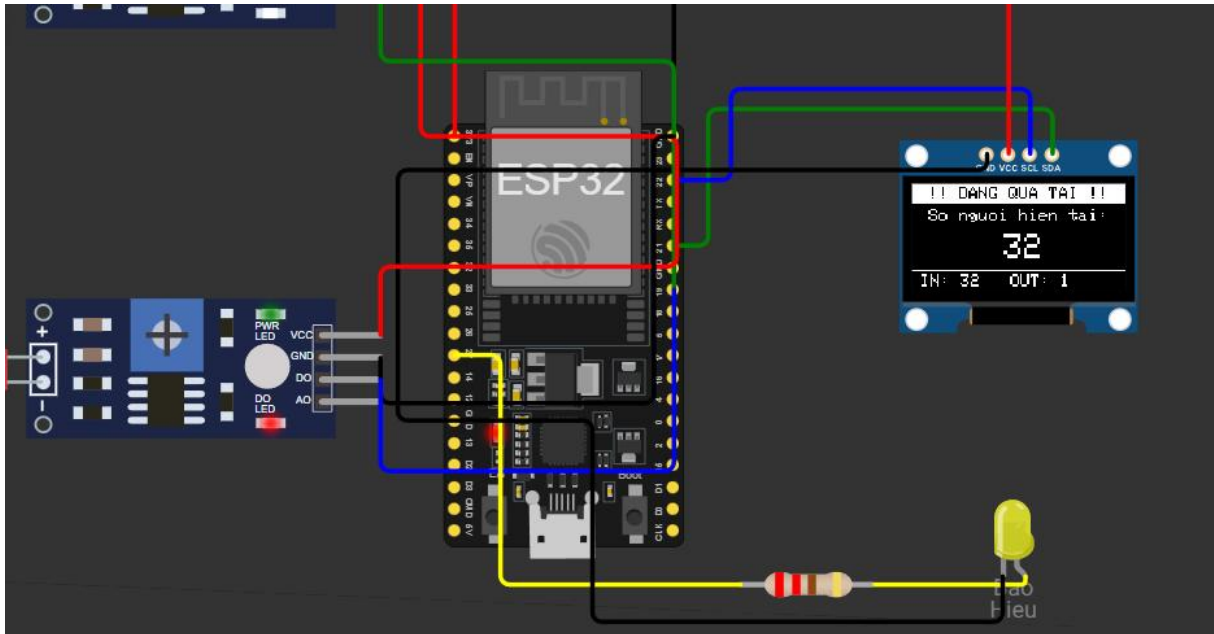
```

```

[FSM] A bi che -> Cho B (Dang Vao)
[FSM] A bi che -> Cho B (Dang Vao)
=> CO NGUOI VAO. Tong vao: 1, Hien tai: 1
Qua thoi gian, huy bo luot di.

```

Hình 12: Serial Monitor hiển thị log chi tiết mỗi sự kiện vào/ra



Hình 13: OLED chuyển sang chế độ cảnh báo khi vượt ngưỡng 30 người

4.2. Nhận xét và đánh giá

Nhìn chung, hệ thống hoạt động đúng theo thiết kế và vượt qua tất cả các trường hợp kiểm thử. Thuật toán FSM tỏ ra đơn giản nhưng hiệu quả – chỉ với 4 trạng thái và một cơ chế timeout, hệ thống xử lý được hầu hết tình huống thực tế. Đây cũng là bài học thú vị về thiết kế phần mềm nhúng: đôi khi giải pháp đơn giản lại là giải pháp tốt nhất.

Ưu điểm của hệ thống:

- Chi phí thấp: Linh kiện phần cứng thực tế ước tính dưới 130.000 VNĐ (ESP32 ~80.000, cặp IR ~10.000, OLED ~20.000, LED và điện trở ~5.000).
- Tiêu thụ điện thấp: Toàn bộ mạch ở chế độ hoạt động tiêu thụ khoảng 100–150 mA @ 5V, có thể chạy bằng pin dự phòng thông thường.
- Khả năng mở rộng: ESP32 có Wi-Fi tích hợp, có thể dễ dàng bổ sung tính năng gửi dữ liệu lên MQTT hoặc HTTP server mà không cần thêm module nào.
- Code sạch, có chú thích: Được tổ chức tốt theo module, dễ đọc và dễ bảo trì.

Hạn chế và điểm cần cải thiện:

- Đếm nhiều người cùng lúc: Hệ thống 2 sensor chỉ đếm được 1 người mỗi lần. Nếu 2 người đi song song qua cửa cùng lúc, chỉ đếm được 1 lần.
- Cảm biến quang trở nhạy với ánh sáng môi trường: Trong phòng có nhiều ánh sáng tự nhiên hoặc ngoài trời, cần che chắn module hoặc điều chỉnh ngưỡng so sánh của mạch LM393 để tránh báo nhầm.
- Không có lưu trữ dữ liệu: Sau khi tắt nguồn, toàn bộ số liệu bị mất. Cần bổ sung SD card hoặc EEPROM để lưu log.

KẾT LUẬN VÀ KIẾN NGHỊ

Kết luận

Qua quá trình thực hiện bài tiểu luận này, em đã hoàn thành được các mục tiêu đề ra ban đầu. Từ một đề tài được nhà trường phân công, em đã chủ động tìm hiểu, nghiên cứu và xây dựng được một hệ thống đếm người hoàn chỉnh trên môi trường mô phỏng Wokwi, lập trình bằng VS Code + PlatformIO. Cụ thể:

- Đã nắm vững lý thuyết về ESP32, cảm biến quang trở (wokwi-photoresistor-sensor) hoạt động theo nguyên lý mô phỏng che tia sáng, màn hình OLED SSD1306 và giao thức I2C thông qua việc tự tìm đọc datasheet và tài liệu kỹ thuật.
- Đã thiết kế sơ đồ mạch mô phỏng (diagram.json) sử dụng component wokwi-photoresistor-sensor, phản ánh đúng nguyên lý quang trở mô phỏng che tia trong toàn bộ code và sơ đồ mạch.
- Đã xây dựng thuật toán máy trạng thái (FSM) phân biệt chính xác chiều vào/ra với cơ chế debounce 40ms, timeout 4000ms và xử lý các tình huống bất thường.
- Hệ thống vượt qua tất cả 6 trường hợp kiểm thử, bao gồm cả các tình huống biên như người quay lại và đếm đến ngưỡng cảnh báo.

Quan trọng hơn, qua bài tiểu luận này em đã rèn luyện được kỹ năng đọc và hiểu tài liệu kỹ thuật bằng tiếng Anh, kỹ năng thiết kế hệ thống nhúng theo mô hình phân lớp, và kỹ năng sử dụng các công cụ phát triển chuyên nghiệp (VS Code, PlatformIO, Wokwi).

Kiến nghị

Nếu có điều kiện phát triển tiếp đề tài này, em đề xuất các hướng cải tiến sau:

- Triển khai phần cứng thực tế: Mua ESP32 DevKit và module cảm biến quang trở LDR kèm mạch so sánh LM393 (module GL5537 phổ biến, ~5.000 VNĐ/cái), lắp ráp và kiểm thử tại cửa phòng thực tế.
- Kết nối IoT: Sử dụng Wi-Fi của ESP32 để gửi dữ liệu lên MQTT Broker, sau đó hiển thị dashboard bằng Node-RED hoặc Grafana.
- Cải thiện độ chính xác: Thêm cảm biến thứ 3 ở giữa để xử lý trường hợp 2 người đi cùng lúc, hoặc nâng cấp lên ESP32-CAM kết hợp nhận diện người.
- Lưu trữ và phân tích: Ghi log vào thẻ nhớ microSD dạng CSV (timestamp, event, count) để phân tích lưu lượng người theo giờ trong ngày.
- Tiết kiệm điện: Ứng dụng Deep Sleep của ESP32 – sau 30 giây không có ai đi qua, hệ thống vào chế độ ngủ, tiêu thụ chỉ ~10 μ A.

TÀI LIỆU THAM KHẢO

Tiếng Việt:

- [1] Nguyễn Đình Phúc (2020). Internet of Things: Nền tảng và ứng dụng. Nhà xuất bản Khoa học và Kỹ thuật, Hà Nội.
- [2] Trần Văn Hùng (2021). Lập trình ESP32 với Arduino Framework. Tạp chí Khoa học và Công nghệ, Đại học Đà Nẵng, tập 19, số 2, tr. 45–52.
- [3] Lê Minh Tân (2019). Ứng dụng cảm biến quang trở trong hệ thống tự động hóa. Kỷ yếu Hội nghị Khoa học Công nghệ Điện tử và Viễn thông, tr. 112–118.

Tiếng Anh:

- [4] Adafruit Industries (2024). Adafruit SSD1306 OLED Displays – Pinouts and Library Guide. <https://learn.adafruit.com/monochrome-oled-breakouts>. Truy cập ngày 15/11/2024.
- [5] PlatformIO (2024). PlatformIO IDE for VS Code – Official Documentation. <https://docs.platformio.org/en/latest/integration/ide/vscode.html>. Truy cập ngày 12/11/2024.
- [6] Wokwi (2024). Wokwi for VS Code – Simulate Your Hardware. <https://docs.wokwi.com/vscode/getting-started>. Truy cập ngày 10/11/2024.
- [7] Espressif Systems (2023). ESP32 Technical Reference Manual v5.1. https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf. Truy cập ngày 08/11/2024.
- [8] Maier, A., Sharp, A., & Vagapov, Y. (2017). Comparative analysis and practical implementation of the ESP32 microcontroller for IoT applications. Proceedings of 5th IEEE International Conference on Internet Technologies and Applications, Wrexham, UK, pp. 143–148.
- [9] Components101 (2022). LDR Photoresistor Datasheet and Working Principle. <https://components101.com/resistors/ldr-datasheet>. Truy cập ngày 20/11/2024.